

Witness-CL: Continuation Contracts for Online Improvement and Retention

Samuel Mausberg

AI-assisted research draft; author review and independent replication pending

Abstract

An agent can preserve its memories while forgetting how to use them, and a locally beneficial change can worsen the future state. We develop a continuation-aware revision of an evidence-preserving online learner. The integrated reference learns from its own executed transitions in a supplied finite deterministic environment family, checks full-horizon Bellman advantages, preserves its previous certified incumbent, and debits worst-case exploration loss before execution. We prove conditional all-state improvement, retention within a truth-preserving contracting model family, and a cumulative exploration-deficit bound. Separate components study closed-state continuation equivalence, fixed-family noise-aware evidence, immutable learned-feature modules, and coupled simulation. On 17,920 synthetic episodes, the budgeted controller matches rather than beats a strong full-history model-based learner’s late return. A 384,000-interaction neural diagnostic shows zero measured forgetting with immutable modules, but replay is a strong alternative and versioning slightly lowers new-task accuracy. We report negative systems results and explicit counterexamples. The release contains 271 passing Python tests, 632 passing C++ cases plus an overflow check, 39 uncompiled Lean theorem attempts, and uncompiled CUDA drafts. It does not establish a native language-agent benchmark win, a general no-forgetting solution, or novelty of the combined design.

1 Problem and scope

The intended deployed learner improves from a sequence of its own episodes without an intervening offline re-training phase. The relevant target is transfer to later problems with shared structure, not merely recalling a transcript or solving an identical instance more often. CL-Bench reports that simple in-context learning (ICL) outperforms dedicated memory systems in its evaluated configurations across expert-validated stateful domains [1]. AgentCL separates controlled compositional streams from naive and held-out streams; the latter can reveal memory-induced degradation [2]. Neither observation implies that online adaptation never works or that ICL is universally optimal.

Our previous revision implements replay-safe rule

growth, recurrence archives, local one-outcome audits, and fixed-feature protected residuals. Its main unresolved issues are continuation credit assignment, exploration loss before admission, protection when features change, and the gap between retaining a rule and retaining the complete deployed behavior. This revision addresses restricted forms of each and records where the resulting mechanism fails to offer an empirical advantage.

The main integrated result is deliberately narrower than a general language agent. We supply the possible finite worlds, but not the true world, and reveal only actual executed transitions. The learner performs online identification, policy improvement, and budgeted experiments. The primitive policy-improvement and concentration arguments are classical. The proposed research direction is an explicit continuation contract linking evidence, state, immutable versions, admission, and execution cost. Its high-impact hypothesis remains unproven: that compact contracts can be learned and reused cheaply enough to beat raw-history ICL.

What is unsolved. A practical learner must discover sufficient state, obtain feedback informative about future consequences, propose useful changes, route them correctly under recurrence or drift, and pay less for learning and verification than it gains in future episodes. A supplied finite environment class bypasses the hardest representation problem. We do not mistake its conditional theorem for an answer to that acquisition problem.

2 Related approaches

Context and memory. Raw-history ICL is a mandatory comparator because it retains information that a premature summary may discard. ACE evolves structured playbooks [3]; ALMA searches executable memory designs through a meta-agent [6]. These approaches concern what is stored, retrieved, and updated, but memory quality alone does not certify downstream behavioral preservation. Meta-design search also has a cost that must be separated from strictly online deployment. AgentCL’s MemProbe stores interactions, insights, and skills with a reliability filter, making it a stronger controlled-transfer comparator than an arbitrary retrieval baseline [2].

Harness evolution. EvoTest updates the agent’s prompt, memory, parameters of the harness, and tool-use routines after episodes without weight fine-tuning [4]. TTHe evolves executable harnesses during evaluation and commits using execution-derived proxy signals; proxy reliability is an explicit challenge [5]. Decision-Aware Memory Cards focuses selection on downstream action utility rather than generic relevance [7]. Our continuation framing does not establish superiority over any of these implementations. It asks for a stronger accounting of persistent state and delayed consequences than a next-action or proxy-score comparison alone.

Conservative learning and neural retention. SPIBB bootstraps from a baseline in uncertain regions using a fixed dataset [11]. StepMix and EpsMix study conservative exploration with episode-wise guarantees in tabular MDPs [13]. DeepSPI already connects online safe policy improvement, world models, and representation learning, with evaluation on ALE-57 [12]. These are direct prior-art constraints: our supplied-world result is narrower, not a first theorem for safe online learning. None of these settings alone establishes retention of heterogeneous language-agent skills under changing memory and routing. Accordingly, our budget is not the first safe exploration mechanism. Progressive neural networks already exploit frozen previous columns [9]; gradient projection protects specified functional directions [10]. We use isolation to address a concrete limitation of a saturated fixed-feature protected subspace, not to claim invention of architectural non-forgetting. Time-uniform confidence methods are likewise established [8].

3 Setting and four retention notions

Let $\mathcal{M}$ be a finite family of stationary deterministic environments with finite states S , actions A , and horizon H . World m has transition $T_m(t, s, a)$ and integer reward $r_m(t, s, a)$. Time and the complete state are observable; the true world index m^* is unknown. Episode boundaries are genuine resets. Every policy p maps (t, s) to an action and has remaining value

$$V_m^p(H, s) = 0, \quad (1)$$

$$V_m^p(t, s) = r_m(t, s, p(t, s)) + V_m^p(t + 1, T_m(t, s, p(t, s))). \quad (2)$$

The original immutable baseline is b_0 . The consistency family $\mathcal{C}_n$ contains worlds agreeing with every actually observed reward and successor transition. Truth membership $m^* \in \mathcal{C}_n$ follows by induction if the family was initially realizable and observations are exact. This is an assumption, not an empirical calibration claim.

We distinguish four protections. *Evidence retention* preserves observations. *Function retention* preserves a

stored module with unchanged input construction. *Incumbent retention* protects the performance of already certified policies. *Deployment retention* must also account for the router, exploration, hidden controller state, and the future environment. The first three alone do not imply the fourth.

Class growth and contradiction. A constraint redundant in the old class need not be redundant after adding new models. All historical witnesses are therefore replayed on every new model. An empty $\mathcal{C}_n$ is a contradiction, never a vacuous certificate. A model-family expansion can invalidate a previous policy comparison and begins a separately marked trust era. Under hidden drift, an adverse outcome may occur before its first contradiction. Detection is not retroactive prevention.

4 Continuation-aware admission

4.1 Full state, not just matching actions

Let a complete system transition be $F(z, u) = (z', o)$, where z includes environment state, controller memory, selected module, input normalization, relevant tool state, and random-generator state; u is an exogenous input. Define G for a replacement system on the same state space.

Theorem 1 (Closed continuation equality). *If a protected predicate P satisfies, for every $z \in P$ and permitted u , $F(z, u) = G(z, u)$ and $F(z, u)_1 \in P$, then every finite input sequence produces the same output trace and final complete state under F and G , when started from the same state in P .*

Proof. The empty sequence is immediate. Equality gives the same first output and successor state. Closure places that successor in P , so induction applies to the remaining input sequence. $\square$

The condition is strong but precise. Equal current actions do not imply equal controller-memory updates. Two controllers can agree now and diverge on the next action. A finite test set is not a proof of closure on an unrestricted state space; the executable finite-domain checker requires an enumerated protected domain.

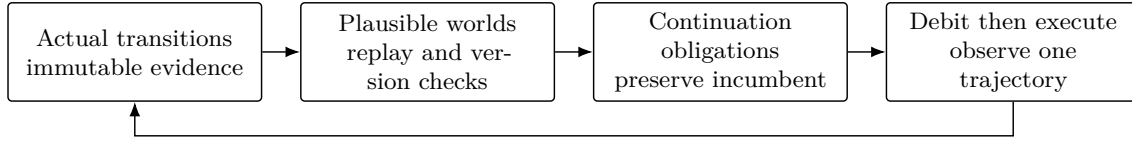
4.2 Robust Bellman obligations

For incumbent b , define the continuation-aware advantage of action a :

$$A_m^b(t, s, a) = r_m(t, s, a) + V_m^b(t + 1, T_m(t, s, a)) - V_m^b(t, s). \quad (3)$$

A candidate policy c passes if

$$\min_{m \in \mathcal{C}_n} A_m^b(t, s, c(t, s)) \geq 0 \quad \text{for every } t, s. \quad (4)$$



Separate tested branches: noisy likelihood evidence; immutable learned modules; coupled simulator.

Figure 1: Implemented deterministic learning loop. Planned model rollouts are not observed outcomes. The independent branches are not yet a unified language-agent system. Learned abstract worlds are the proposed next step, not a feature supplied by this finite-world experiment.

The incumbent’s own action is always admissible. The implementation maximizes summed model advantages among admissible actions and prefers the incumbent on ties. That ranking is a heuristic; the guarantee uses the minimum.

Theorem 2 (All-state improvement). *If $\mathcal{C}_n \neq \emptyset$ and (4) holds, then $V_m^c(t, s) \geq V_m^b(t, s)$ for every $m \in \mathcal{C}_n, t, s$.*

Proof. Terminal values agree. At an earlier stage, apply induction at the candidate’s successor and then (4):

$$\begin{aligned}
 V_m^c(t, s) &= r_m(t, s, c) + V_m^c(t+1, T_m(t, s, c)) \\
 &\geq r_m(t, s, c) + V_m^b(t+1, T_m(t, s, c)) \\
 &\geq V_m^b(t, s).
 \end{aligned}$$

□

Theorem 2 is classical policy-improvement reasoning. It is a sufficient, not necessary, admission rule. Some coordinated policy changes improve full value even though an intermediate action has negative advantage relative to the old continuation. More expressive policy-level certificates could admit them. An average-return criterion would be cheaper but could sacrifice a protected rare state, so it is not substituted for the stated all-state guarantee.

4.3 Protect acquired gains, not only the anchor

Within a contracting family, every previous universal obligation remains true. Consequently, each new improvement round begins from the *previous certified incumbent*. By transitivity and truth membership, its true all-state value is nondecreasing. Rebuilding from b_0 each episode protects only the original floor; it can forget an earlier improvement. We found and corrected precisely that bug while developing this revision, then added update-by-update regression tests.

Contracts bind the baseline hash, candidate hash, model-family identities, and generation. A changed encoder, model family, baseline, or scope requires a new contract. The implementation rejects stale or foreign certificates. After new plausible worlds are admitted, it

Algorithm 1: Deterministic continuation-contract learner. No step uses the evaluator’s true-world index.

Input: Family $\mathcal{C}$, incumbent b , anchor b_0 , archive, spent budget L , cap B
 Replay new observed transitions; reject certification if $\mathcal{C} = \emptyset$
 If plausible models were added, start a new trust era with $b \leftarrow b_0$
 Improve b using all-model, all-state Bellman obligations
 Construct candidate trials from model optima and the archive
 Compute predicted signatures and $D(p)$; discard trials with $L + D(p) > B$
 Select an informative feasible trial p , or use b
 Reserve $L \leftarrow L + D(p)$; execute p once
 Record the actual complete trajectory; update $\mathcal{C}$ and archive

starts a new trust era from b_0 rather than claiming that earlier incumbent guarantees extend to a larger family.

5 Accounting for exploration

A safe incumbent need not generate identifying evidence. We therefore examine candidate episode policies from surviving model optima and the immutable archive. For candidate p at actual initial state s_n , reserve

$$D_n(p) = \max \left\{ 0, \max_{m \in \mathcal{C}_n} [V_m^{b_n}(0, s_n) - V_m^p(0, s_n)] \right\}. \quad (5)$$

Run the trial only if past debits plus $D_n(p)$ do not exceed the declared budget B . Reserve this debit *before* execution and never refund it merely because the realized trial was favorable. Among feasible trials, the reference prioritizes worst-case reduction in the plausible-family size, then optimistic gain and lower debit. Predicted trajectories are computed from supplied models; only the actual executed trajectory is admitted as evidence.

Theorem 3 (Completed-prefix deficit). *In a stationary realizable deterministic era with genuine episode resets,*

Algorithm 1 satisfies, at every completed episode prefix N ,

$$\sum_{n=1}^N [V_{m^*}^{b_0}(0, s_n) - R_n] \leq \sum_{n=1}^N D_n \leq B. \quad (6)$$

Proof. Incumbent improvement gives $V^{b_0} \leq V^{b_n}$. Truth membership and the maximization in (5) give $V^{b_n} - V^{p_n} \leq D_n$ at s_n . Deterministic execution gives $R_n = V^{p_n}$. Add these inequalities. The admission ledger ensures the second inequality. $\square$

Initial states need not be independently sampled. They must be the same actual states used for the baseline comparison. With cross-episode carryover, Equation (6) is not a comparison to the baseline’s counterfactual lifelong trajectory: prior actions may change later starts. One must include the missing continuation value or prove regeneration before making that stronger claim. The budget also does not prohibit individual adverse actions, privacy violations, or irreversible real-world harm.

Proposition 4 (A zero-risk obstruction). *Consider two observationally indistinguishable worlds in which the baseline always returns 5, while a trial returns 0 in one and 10 in the other. No algorithm required to have zero expected deficit in both worlds can assign positive trial probability before obtaining distinguishing information.*

Proof. A trial probability $p > 0$ produces expected deficit $5p$ in the unfavorable world. Baseline observations cannot distinguish the worlds. Thus universal zero expected deficit requires $p = 0$, preventing strict improvement in the favorable world. $\square$

This does not end the research problem. It identifies the resource that must be provided: safe informative actions, a nonzero exposure allowance, trusted side information, or a legally available simulator.

6 Independent extensions

6.1 Fixed-family noise-aware evidence

Hard deletion is incorrect when valid hypotheses sometimes produce an unexpected outcome. For a fixed finite probabilistic family and prior weights $\pi_m > 0$ chosen before data, let $L_{m,n}$ be the likelihood of actual observations under predictable chosen actions and let $L_{\text{mix},n} = \sum_m \pi_m L_{m,n}$. The ratio $E_{m,n} = L_{\text{mix},n}/L_{m,n}$ is a nonnegative supermartingale under true m ; common-support conditions give martingale equality. Standard maximal-inequality reasoning yields [8]

$$\mathbb{P}_m\{\exists n : E_{m,n} \geq 1/\delta\} \leq \delta. \quad (7)$$

The exact-rational reference retains m when $\delta L_{\text{mix},n} < L_{m,n}$. Tests enumerate all binary histories of length six

and verify the likelihood-ratio expectation exactly in a common-support example.

This does not justify assigning a new data-dependent model retrospective likelihood credit. Nor does it turn an uncalibrated neural score into a likelihood. The filter is not integrated into the deterministic controller. A stochastic controller would additionally need expected-value backups and a valid allowance for realized-return deviations. It cannot inherit (6) by renaming a consistency set a confidence set.

6.2 Learning features without overwriting them

The earlier fixed-feature residual protects an old feature span but loses freedom when that span fills the representation. The new neural component trains both features and a head in a separate module, then freezes the resulting arrays as immutable bytes. Old public-schema dispatch and old input construction do not change. Accordingly, the old function is unchanged for every input still routed to that module. This is architectural isolation, related to progressive networks [9], with growing storage rather than a fixed-memory solution. Functional preservation becomes full behavioral preservation only when the closed-state conditions of Theorem 1 are also satisfied.

The executed diagnostic has two exhaustive actions and exact success feedback. Given the chosen action, success or failure determines the binary target. This own-feedback supervision argument is invalid for noisy outcomes or arbitrary multi-action tasks. Every baseline receives the same public scope tag. No latent scope inference, LLM weight update, or general transfer result is established.

6.3 Coupled simulator execution

The paired simulator advances two controller memories independently under a shared exogenous noise tape. It shares one environment transition exactly when the two actual states and actions agree, otherwise evaluates both. Actual reconvergence can permit later sharing; matching now never licenses skipping an unchecked suffix. For H steps and L shared transitions, the environment-call count is $2H - L$ instead of $2H$, while controller calls remain $2H$. This is a semantic work-saving identity, not a claim that arbitrary external tools can be cloned or that LLM serving becomes proportionally faster.

7 Experiments and negative results

7.1 Stateful online learning

We ran twenty independent seeds, sixty-four episodes, seven arms, and two finite world families: delayed damage and compositional navigation. Each has $H = 5$, $|S| = 4$, $|A| = 3$, and a supplied family constructed from sixteen latent masks; semantically duplicate worlds are deduplicated. The hidden mask is not given to the learner. Episodes reset and initial states cycle through the public state space. All methods receive the same actual feedback protocol. The complete 17,920 episode records are archived with seed-level summaries.

The strong full-history optimistic baseline uses all observed transitions to filter the same supplied family and plans optimistically. It is not a language model and must not be labeled raw-history ICL. Uniform exploration followed by planning and immediate-reward greedy control isolate exploration and continuation effects. Our $B \in \{0, 4, 16\}$ arms isolate conservatism. Oracle values are computed only by the evaluator for diagnostics; they are not extra training feedback.

Table 1 supports the continuation mechanism but rejects a performance-breakthrough interpretation. In delayed damage, immediate greedy control has worst cumulative deficit 133, versus 2 for $B = 4$. However, the strong full-history baseline also has worst deficit 2 and reaches exactly the same late return as $B = 4$ on every seed in both domains. The paired late-return difference is identically zero in this study. Zero budget reduces delayed-damage late return from 13.1375 to 12.5625. The contract controller is also slower on these small CPU problems because it performs additional all-state and trial planning work. Its guarantee, rather than a measured performance advantage, is the positive result here.

The delayed-damage seed-level late-return standard deviation is 0.620 for the full-history and $B = 4$ methods, and 0.628 for $B = 0$. Episodes are correlated within a seed; treating 17,920 rows as independent observations would overstate statistical evidence. Generalization is limited by two small generated families, supplied model structure, exact feedback, and short horizons.

7.2 Online feature learning and retention

Twenty seeds cover three public scopes with 1,600 feedback examples each and four arms, totaling 384,000 online interactions. Shared and versioned networks use the same five input features: two numeric observations plus a public three-way scope tag. Each scope has 512 independent held-out probes. New-task online accuracy is measured before each update. Forgetting is the acquired held-out accuracy minus later accuracy, averaged across

old-scope checkpoints. No separate offline training phase is inserted.

Table 2 includes an online reservoir-replay baseline rather than relying only on naive shared SGD. Replay reduces mean forgetting to 2.65 percentage points. Versioned learned modules have exactly zero measured forgetting and old-logit drift, but late current-task accuracy is 95.43%, versus 96.23% for replay. Thus the experiment does not show a free stability-plasticity gain. The tiny parameter footprints describe this toy network, not an LLM deployment. Raw per-update neural rows are not archived; fixed generation seeds, code, and per-seed results are supplied for reproduction.

7.3 Noise, sharing, and kernel costs

The noise experiment draws 1,000 independently seeded length-64 sequences with 10% noise and $\delta = 0.05$. The fixed-family likelihood filter ever excludes truth in 12 sequences (1.2%); hard deletion loses truth in 997 (99.7%). The finite test does not prove coverage. Correct likelihood specification remains an explicit premise of the probability argument.

Across 200 paired simulator tests of horizon 32, every coupled trace matches its independent reference. Mean environment calls fall from 64 to 39.405, a 38.43% reduction. Controller calls remain 64. Late-divergence construction favors sharing; this does not estimate savings in real tool or LLM workloads.

The packed NumPy continuation pass has essentially no cold end-to-end advantage in the measured shapes. At $M = 4, 16, 64$, scalar times are 0.885, 2.792, 9.830 ms, versus packed 0.809, 2.725, 10.024 ms. Medians include table construction and value computation. Packing is slightly slower at 64 models. Device-resident caching may change that outcome, but no such measurement exists in this release.

8 Engineering and verification

One dense all-model, all-action advantage pass costs $O(MHSA)$, with model storage of the same order. The implementation evaluates up to the number of unique model optima plus archived candidates; scoring K candidates by predicted signatures costs $O(KMH)$ after their construction, and value computations add work. Repeated policy-improvement passes multiply admission cost. Model-family enumeration is not cheap simply because its arithmetic can be vectorized.

The checked C++ scalar reference uses wide intermediates and rejects int64 overflow. The NumPy primitive rejects unsafe magnitude bounds before arithmetic. The CUDA draft proposes one 256-thread block per (t, s, a) to reduce across model advantages. It assumes valid indices, a positive model count, matching immutable model/value snapshots, and the documented integer

Method	Delayed damage			Compositional navigation		
	Early	Late	Worst deficit	Early	Late	Worst deficit
Frozen anchor	8.750	8.750	0	6.250	6.250	0
Immediate-reward greedy	8.150	8.150	133	9.312	9.312	0
Full-history optimistic	12.869	13.137	2	11.012	11.262	0
Uniform explore then plan	10.900	13.137	19	10.131	11.262	4
Continuation, $B = 0$	12.275	12.562	0	10.925	11.262	0
Continuation, $B = 4$	12.850	13.137	2	11.000	11.262	0
Continuation, $B = 16$	12.806	13.137	2	11.006	11.262	0

Table 1: Executed finite-world experiment, means over twenty seeds. Early and late are the first eight and final sixteen episodes. Worst deficit is the maximum cumulative anchor-relative deficit over all completed prefixes and seeds, in native reward units. This is a model-based mechanism study, not an ICL or LLM benchmark. Seed-level variability is retained in the result ledger.

Method	Early (%)	Late (%)	Forgetting (pp)	Parameter bytes	Updates/seed
Shared, equal width	64.72	96.13	47.09	1352	4800
Shared, near-matched parameters	68.88	96.26	44.58	4040	4800
Shared, online replay	70.03	96.23	2.65	4040	9599
Immutable learned modules	74.87	95.43	0.00	4056	4800

Table 2: Executed learned-feature diagnostic, means over twenty seeds. Early and late use the first 100 and last 200 examples per scope. Forgetting is in percentage points. The near-matched shared model has 4,040 parameter bytes versus 4,056 for three immutable modules. Replay additionally stores 10,752 payload bytes, excluding Python overhead, and approximately doubles online updates.

bound. There is no executed host binding, sanitizer result, or GPU timing for that draft. A useful next optimization would retain immutable tables and generation-indexed value caches on device, but only if profiling shows this work matters after model fitting, LLM inference, and tool execution are included.

Validation status. All 271 Python tests pass, including delayed harm, hidden controller divergence, empty-family rejection, stale certificates, class expansion, prefix deficits, incumbent retention, noisy evidence, immutable arrays, and scalar/vector parity. All 432 legacy C++ cases and 200 new randomized cases pass, plus an explicit overflow rejection test. The new C++ test runs with undefined-behavior checking. These observations are not proofs of unrestricted implementation correctness.

The repository has 39 Lean theorem declarations: thirteen original, sixteen in the prior refinement, and ten continuation declarations. All are attempts, not kernel-checked theorems. The local build fails because `lake` is absent, and the official toolchain download attempt also fails. The new source addresses closed traces, Bellman comparison, truth membership, shrinking obligations, integer debit addition, a zero-risk obstruction, and incumbent ordering. It does not formalize stochastic coverage, floating point, Python refinement, or CUDA. The included `#print axioms` commands are unexecuted source, not evidence that a compiler checked their dependencies.

9 Learning compact abstractions

The strongest untested extension is counterexample-guided abstraction refinement. A deployed learner would construct a typed abstract state from observable schema and raw witnesses, synthesize a skill, and split abstract states when different future consequences were incorrectly merged. Earlier abstractions and modules remain versioned. New candidates need either a uniform model-error certificate or an independent prospective full-continuation audit. Training fit alone is not such a certificate.

Proposition 5 (Uniform approximate-model margin). *Suppose a true and abstract finite-horizon model share a valid policy/state correspondence. Both reward functions lie in $[0, R_{\max}]$, their absolute reward discrepancy is at most ϵ_r , and their one-step transition total variation is at most ϵ_p , uniformly on all relevant state-action pairs. For any fixed policy with h remaining steps, the value error is at most*

$$E_h = h\epsilon_r + R_{\max}\epsilon_p \frac{h(h-1)}{2}. \quad (8)$$

An abstract candidate-minus-baseline advantage of at least $2E_h$ therefore implies nonnegative true value difference at that state.

Proof. The terminal error is zero. Each recursion adds at most ϵ_r reward error, E_{h-1} successor value error, and $\epsilon_p(h-1)R_{\max}$ distribution error, because the successor

value span is at most $(h - 1)R_{\max}$. Summing proves E_h . Apply the error bound to both policies and subtract. Adaptive policy selection requires a simultaneous envelope covering all policies that may be selected. $\square$

This is a useful rejection rule, not a claim to have learned the required uniform envelope. Partial observability can make an abstraction non-Markovian; rare states can invalidate empirical coverage; longer horizons inflate the bound quadratically; and conservative margins can reject nearly every useful change. The first experiment should hide the generator family from the learner while keeping the exact world available to the evaluator, then independently vary aliasing, delayed rewards, horizon, grammar depth, and misspecification. Compare learned abstractions with supplied models, full-history planning, and a real retrieval-plus-LLM learner on held-out composition and recurrence.

Memory cannot be free. After N independent binary facts, exact answers to arbitrary old-fact queries require distinguishing 2^N possible histories, hence at least N bits of system state. Bounded active memory plus an external archive is compatible with this limit; uniformly bounded total storage and unlimited exact arbitrary recall are not. Learned abstractions must compress reusable structure, not claim to compress independent information without loss.

10 External evaluation plan

No native CL-Bench, AgentCL, J-TTL, model-server, or GH200 run was executed. The current CL-Bench ICL source was inspected: it provides respond/observe/reset, usage logging, and FIFO context handling, but not the finite environment family required by our controller. A generic wrapper is not a completed native adapter. The repository records inspected file hashes and an integration protocol.

Use two fixed backbones, all six CL-Bench domains, and AgentCL’s controlled, naive, and held-out streams. Mandatory applicable comparators include stateless, raw-history ICL, scratchpad ICL, ACE, Mem0, MemProbe, harness evolution, v0.2, and the proposed method. Charge all generated candidates, judges, replay, online updates, model fitting, failed verification, and simulator work. Compare both equal token/cost budgets and equal wall time. Use at least twenty paired independent seeds per domain where allowed and resample whole streams, not correlated episodes. A development-only power pilot should set a final sample size before test inspection.

The proposed practical target is at least five percentage points higher native learning gain, a positive paired 95% confidence lower bound in the prespecified pooled comparison, positive point estimates on at least four of six CL-Bench domains, and no protected domain regression beyond two percentage points. These are chosen research thresholds, not consequences of our theorems. Simultaneous per-domain claims require appropriate uncertainty correction. A nonsignificant loss is not evidence of noninferiority.

Kill the claimed learning advantage if the strongest cost-matched baseline matches or wins inside the prespecified practical-equivalence margin. Kill the abstraction mechanism if honest coverage rejects over 90% of otherwise useful proposals, state size grows with raw history rather than structure, or held-out aliasing breaks a claimed guarantee. Kill the systems speed claim if it disappears after packing, model fitting, launches, and cache management are charged. Any reproducible exact mismatch ends the associated correctness claim immediately.

Resources. The included mechanisms require CPU, NumPy, pytest, and a C++17 compiler. A single GH200 is a proposed open-model test platform, subject to the selected model’s actual weight, cache, activation, optimizer, and module footprint. Native CL-Bench documents Python 3.13+, uv, and Docker. Paired execution requires legal, read-only snapshotable workspaces. As an explicit planning example, six domains, two backbones, seven arms, twenty seeds, and sixty-four episodes yield 107,520 agent episodes before extra candidates. At an assumed 20,000 processed tokens per episode, this is 2.1504 billion tokens. Neither this assumption nor any actual GPU speed or native run duration was measured; use a pilot for resource budgeting.

11 Conclusion

This revision turns vague retention language into specific continuation, incumbent, and exploration contracts. The finite-world learner improves online without an offline retraining phase and has conditional retention and deficit proofs. The experiments support the mechanisms but do not beat a strong full-history model-based baseline, establish a cost-free neural retention gain, or demonstrate native language-agent superiority. The consequential open task is learning compact, reliable abstractions and useful reusable programs whose verification costs less than the future value they create. The artifact makes that hypothesis executable and falsifiable rather than presenting it as solved.

Research and safety statement. Prepared as an AI-assisted research draft for Samuel Mausberg. Independent author review, literature assessment, formal checking, and replication are required. All executed environments are pure numeric simulators. Do not authorize irreversible real-world actions from these reward-model certificates. A deployed implementation must isolate untrusted tool content and prevent proposed skills from altering evaluators, permissions, history, or the cost ledger. The artifact is MIT licensed and preserves the supplied earlier Git history.

References

- [1] Parth Asawa, Christopher M. Glaze, Gabriel Orlanski, Ramya Ramakrishnan, Benji Xu, Asim Biswal, Vincent Sunn Chen, Frederic Sala, Matei Zaharia, Joseph E. Gonzalez. Continual Learning Bench: Evaluating Frontier AI Systems in Real-World Stateful Environments. 2026. <https://arxiv.org/abs/2606.05661>.
- [2] Yiheng Shu, Bernal Jiménez Gutiérrez, Saisri Padmaja Jonnalagedda, Yuguang Yao, Huan Sun, Yu Su. AgentCL: Toward Rigorous Evaluation of Continual Learning in Language Agents. 2026. <https://arxiv.org/abs/2606.02461>.
- [3] Qizheng Zhang, et al. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. 2025. <https://arxiv.org/abs/2510.04618>.
- [4] Yufei He, et al. EvoTest: Evolutionary Test-Time Learning for Self-Improving Agentic Systems. 2025. <https://arxiv.org/abs/2510.13220>.
- [5] Jun Nie, et al. TTHE: Test-Time Harness Evolution. 2026. <https://arxiv.org/abs/2607.08124>.
- [6] Yiming Xiong, Shengran Hu, Jeff Clune. Learning to Continually Learn via Meta-learning Agentic Memory Designs. 2026. <https://arxiv.org/abs/2602.07755>.
- [7] Xinyu Guan, Qianyang Zhao, Yuming Deng. Decision-Aware Memory Cards: Counterfactual-Inspired Context Selection and Compression for Tool-Using LLM Agents. 2026. Version 3, September 4. <https://arxiv.org/abs/2606.08151>.
- [8] Steven R. Howard, Aaditya Ramdas, Jon McAuliffe, Jasjeet Sekhon. Time-uniform, Nonparametric, Nonasymptotic Confidence Sequences. *The Annals of Statistics*, 49(2):1055–1080, 2021. <https://arxiv.org/abs/1810.08240>.
- [9] Andrei A. Rusu, et al. Progressive Neural Networks. 2016. <https://arxiv.org/abs/1606.04671>.
- [10] Mehrdad Farajtabar, Navid Azizan, Alex Mott, Ang Li. Orthogonal Gradient Descent for Continual Learning. In *International Conference on Artificial Intelligence and Statistics*, 2020. <https://proceedings.mlr.press/v108/farajtabar20a.html>.
- [11] Romain Laroché, Paul Trichelair, Rémi Tachet des Combes. Safe Policy Improvement with Baseline Bootstrapping. In *International Conference on Machine Learning*, 2019. <https://proceedings.mlr.press/v97/laroché19a.html>.
- [12] Florent Delgrange, Raphael Avalos, Willem Röpke. Deep SPI: Safe Policy Improvement via World Models. In *International Conference on Learning Representations*, 2026. <https://arxiv.org/abs/2510.12312>.
- [13] Donghao Li, Ruiquan Huang, Cong Shen, Jing Yang. Near-optimal Conservative Exploration in Reinforcement Learning under Episode-wise Constraints. In *International Conference on Machine Learning*, 2023. <https://arxiv.org/abs/2306.06265>.